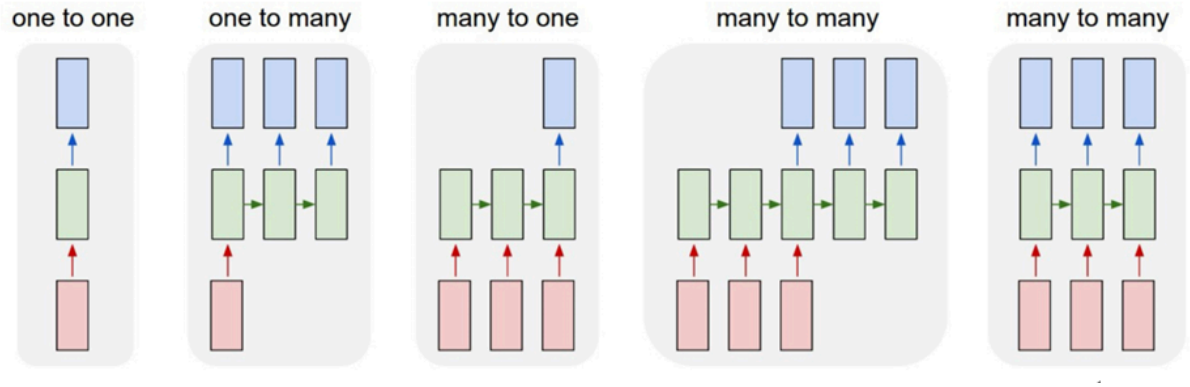
7. Recurrent Neural Networks

> 흐름: **Sequence Modeling → RNN 상태 갱신 → Computation Graph, BPTT → 문자 단위 language model → image·language 응용 → multi-layer RNN → Gradient 문제 → LSTM → 현대 RNN**

1. Sequence Modeling

기본 구조

구조	의미	예시
One-to-one	입력 1 → 출력 1	기존 NN
One-to-many	입력 1 → 출력 sequence	image captioning
Many-to-one	입력 sequence → 출력 1	영상 frames → 행동 class
Many-to-many	입력 sequence → 출력 sequence	영상 → 설명 문장
Many-to-many	각 입력 시점 → 대응 출력	영상의 frame별 분류



2. RNN(Recurrent Neural Network)

내부 상태(h)가 있는 신경망. 이전 정보를 상태에 담아 다음 시점으로 전달.

구조: 현재 입력 + 이전 상태 → 새로운 상태 → 출력

- new state: 과거 정보 + 현재 입력. $h_t = f_{W_{hh}}(h_{t-1}, x_t)$
- output: 상태 → 출력 dimension 변환. $y_t = f_{W_{hy}}(h_t)$

Block 재사용: 모든 시점에서 parameter W 공유

Vanilla RNN / Elman RNN

state: 1개 hidden vector h 사용

$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t),\ y_t = W_{hy}h_t$

- W_{hh}: h → h. W_{xh}: x → h. W_{hy}: h → y.
- tanh 출력 범위: **[−1, 1]**

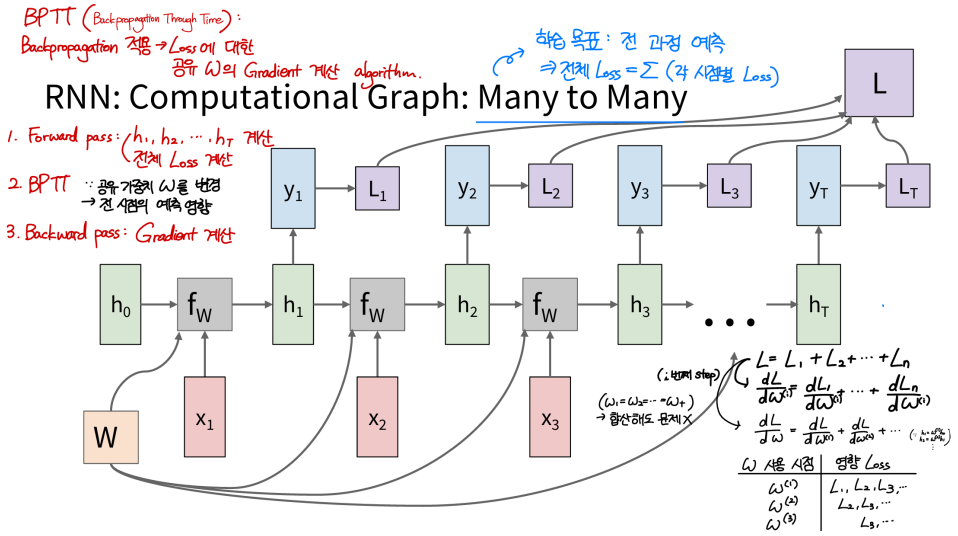
---

3. RNN — Computation Graph, BPTT

3-1. Many-to-many

앞 시점의 상태 → 뒤 시점의 상태, 출력에 영향을 줌.

⇒ Gradient: **현재 출력에 대한 영향 + 이후 출력들에 대한 영향** 반영됨.



Loss: $L = \sum_{t=1}^{T} L_t$

1. Forward pass: h_t, L_t 계산
2. **BPTT**(시간축으로 펼친 computation graph에 backpropagation 적용)
3. Backward pass: 공유 weight에 대한 Gradient = **모든 사용 시점의 기여 합산**.

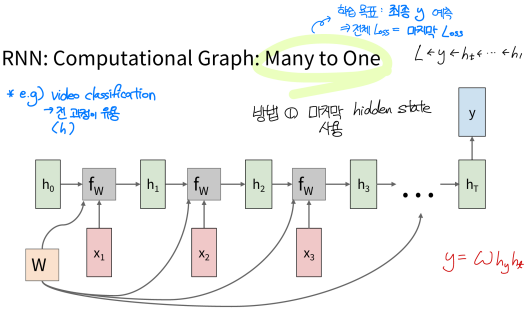
3-2. Many-to-one

y 도출 방식 상이

방법 ① 마지막 hidden state 사용

앞선 입력들이 마지막 상태에 반영되도록 학습.

$$y = W_{hy} h_T$$

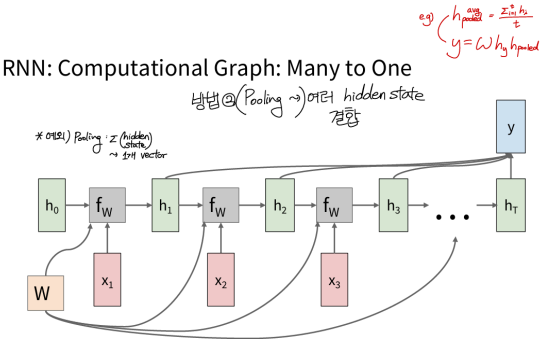


방법 ② 여러 hidden state 결합

(예시) average Pooling:

$$h_{\text{pooled}} = \frac{1}{T} \sum_{t=1}^{T} h_t$$

$$y = W_{hy} h_{\text{pooled}}$$



3-3. One-to-many

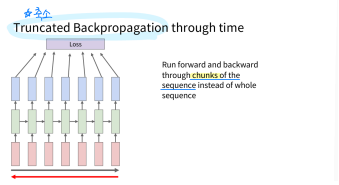
첫 input: x. 이후 input: 0(zero) or y_{t-1}

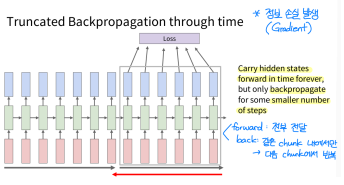
Truncated BPTT — backpropagation범위 제한

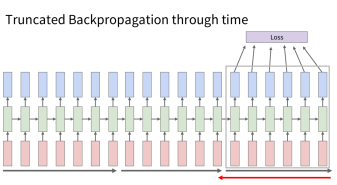
기존: 전체 forward → loss 계산 → 전체 backward ⇒ **memory-bound**.

Truncated BPTT: 긴 sequence → **짧은 구간(chunk)** 분할.

⇒ 과거 상태는 전달되지만, Gradient가 거슬러 올라가는 범위는 제한.







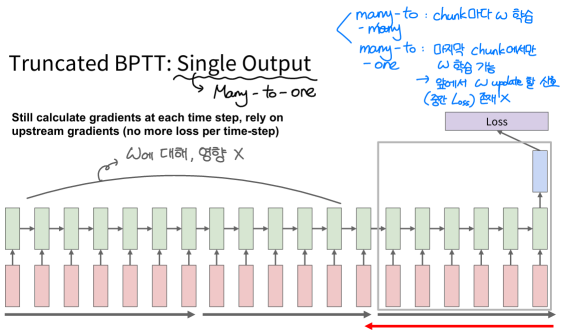
- 과정: **(1개 chunk 내) forward → loss → back(propagation) 수행**. 이후 chunk 수만큼 반복.

  forward: hidden state를 끊김 없이 계속 전달.

  back: 같은 chunk 내에서만 수행.

  - chunk boundary를 넘는 먼 과거의 계산 ← Gradient 전달 X.
  - 먼 과거가 현재 손실에 미친 영향을 학습하는 데 제약.

- **Many-to-One:** 마지막 chunk를 제외한 구간은 자체적인 학습 신호 X.

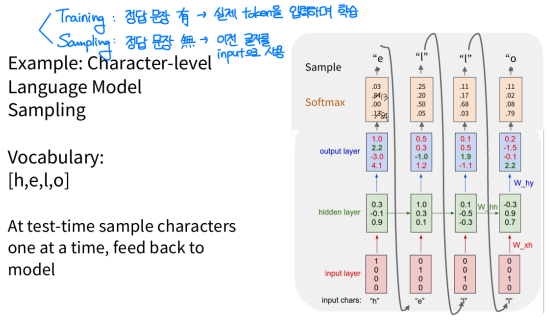


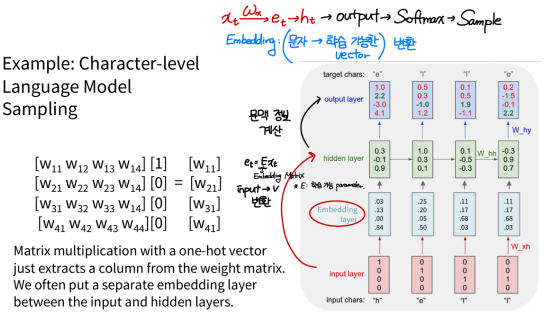
---

4. Character-level Language Model

현재까지의 문자로 다음 문자 예측

- **Training:** 실제 문자열의 문자를 입력. 다음 실제 문자를 정답으로 사용.
- **Sampling:** input → 확률분포 softmax(logit) → y_t 생성. 이를 **다시 model에 입력** 후 반복.





구조

- $x_t \rightarrow e_t = Ex_t \rightarrow h_t = \tanh(W_{hh}h_{t-1} + W_{xh}e_t)$ ⇒ output ⇒ softmax ⇒ sample
- Embedding Layer(e): letter→ learnable vector

  One-hot 벡터와 행렬의 곱 = **해당 문자에 대응하는 열 하나를 선택**하는 연산.
- 같은 문자를 입력해도, 이전 상태가 다르면 다음 문자 예측이 달라질 수 있음.

생성 방식

- **Greedy decoding:** 매 시점 가장 확률이 높은 문자·토큰 선택.
- **Sampling:** 확률분포에서 무작위 선택.

- **Beam search:** 여러 후보 sequence 유지 → sequence 점수 비교.

---

5. RNN의 장단점

장점

- 다양한 길이의 입력 처리 가능.
- 이론적으로 먼 과거의 정보를 현재 계산에 활용 가능.
- 입력이 길어져도 **파라미터 수는 증가하지 않음**. 모든 시점에 동일한 가중치를 적용.

한계

- 이전 상태를 계산해야 다음 상태를 계산할 수 있어, **순차 계산이 느림**.
- 실제로는 먼 과거의 정보를 활용하기 어려움.
- hidden state의 size 고정 → 긴 length을 제한된 벡터에 담아야 함.

⇒ **입력을 계속 받을 수 있다는 것과, 과거 정보를 모두 정확히 기억한다는 것은 구분**.

---

6. 예시

Image Captioning

image→ **CNN** (시각 정보 추출) → **RNN** (단어 생성) → **word sequence**

- **RNN 연결 과정**
  1. CNN의 최종 classification layer 제거.
  2. image → feature vector 추출. $v = \mathrm{CNN}(I)$
  3. RNN의 hidden state 계산에 반영. 이후 hidden state를 통해 전달.
     $h = \tanh(W_{xh}x + W_{hh}h_{\text{prev}} + W_{ih}v)$
- **〈START〉·〈END〉 token:** 문장의 시작·끝 표현.
- 생성 결과 ⇒ **대상 인식과 행동·관계 해석에서 오류**가 발생할 수 있음.

Vision·Language task

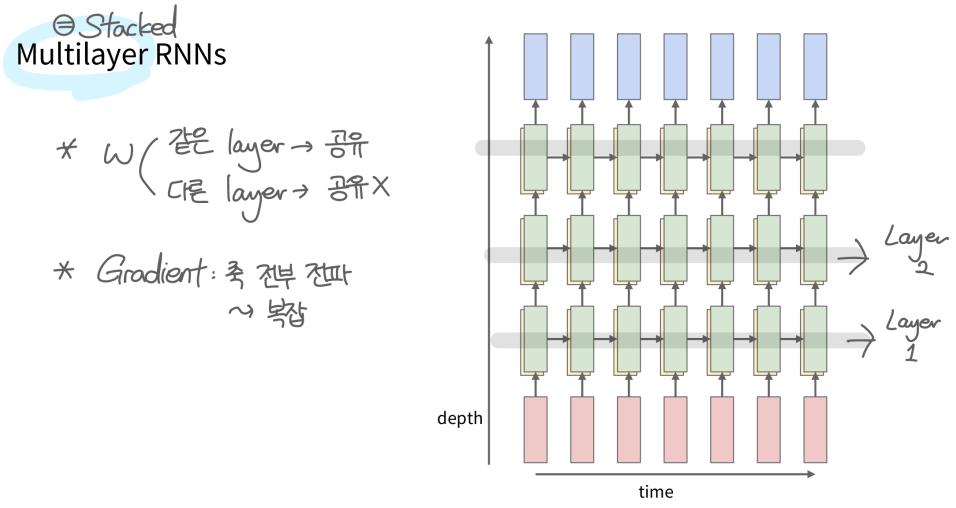
과제	입력·출력
VQA: Visual Question Answering	image + query → answer
Visual Dialog	image에 관한 여러 차례의 query·answer

과제	입력·출력
Vision-Language Navigation	언어 지시 + 변화하는 시각 입력 → 이동 행동 sequence

7. Multi-layer RNNs

가로 방향: time, 세로 방향: depth.

$$h_t^{(\ell)} = f_{W^{(\ell)}}\left(h_{t-1}^{(\ell)}, h_t^{(\ell-1)}\right)$$

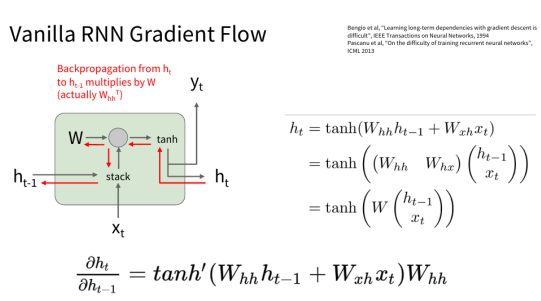


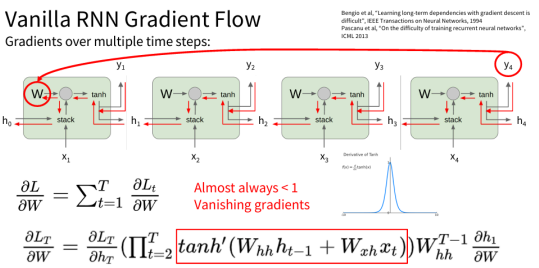
- 전파 방향: layer, 시간 축 모두 전달.
- **같은 층: weight 공유. 다른 층: 별도 weight 사용.**

8. LSTM(Long Short-Term Memory)

Vanilla RNN 문제

이전 시점으로 gradient flow 진행 → **tanh 미분 * 순환 weight 행렬**





⇒ 긴 시퀀스에서 안정적으로 정보를 전달하는 순환 가중치를 학습하기 어려움.

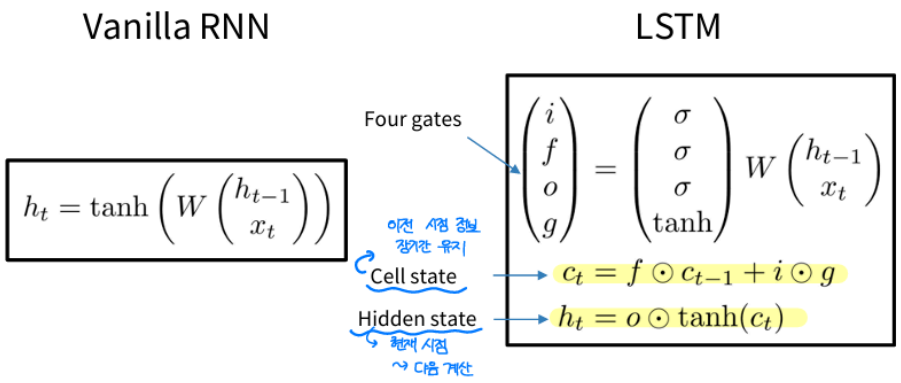
- Largest singular value < 1 → Exploding gradients.

  대응: Gradient Clipping
- Largest singular value > 1 → Vanishing gradients.

  대응: architecture 변경. (대표적인 구조: **LSTM**.)

LSTM

cell 상태를 다음 시점으로 직접 전달할 경로를 둠.



cell 상태 c: 시간에 따라 정보를 유지·갱신하는 경로.

$c_t = f_t \odot c_{t-1} + i_t \odot g_t$

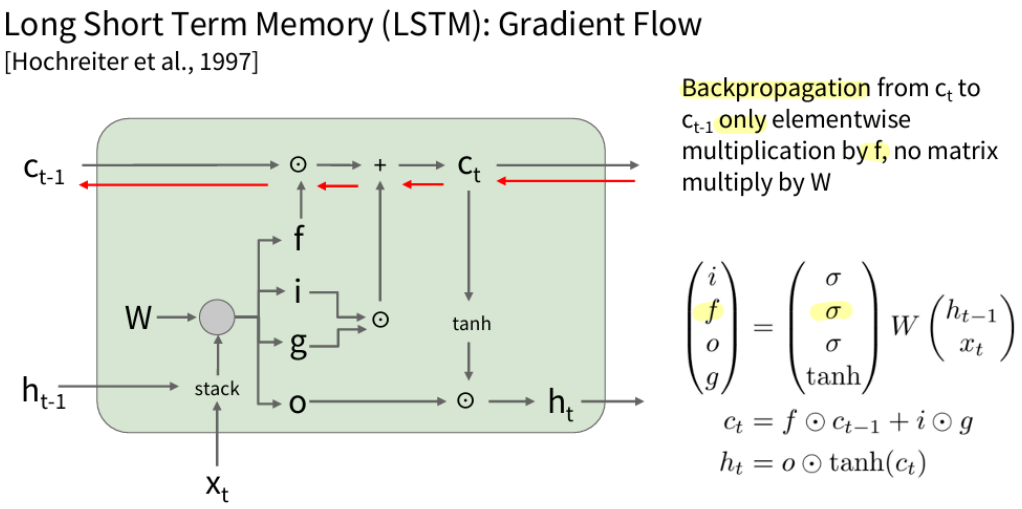
기호	역할
g — 후보 내용	새 정보를 저장할 값, 부호
i — Input gate	새 정보를 얼마나 저장할지
f — Forget gate	기존 정보를 얼마나 유지할지
o — Output gate	셀 정보를 h에 얼마나 드러낼지

hidden state h: 현재 시점에 외부로 드러낼 표현.

$h_t = o_t \odot \tanh(c_t)$

Gradient Flow

- 매 시점 **W를 행렬 곱하는 과정이 없음**.
- Forget gate와 원소별 곱을 거쳐 전달.
- **기울기 소실·폭발이 전혀 없음을 보장하지는 않음**.



ResNet과의 연결

⇒ 정보를 전달하는 경로를 구조적으로 마련한다는 공통점.

- ResNet도 기존 표현이 여러 변환을 거치지 않고 전달될 경로를 둠.
- **Highway Networks**

---

10. Modern RNNs — 상태 공간 모델로의 연결

sequence 길이에 따라 계산량이 linearly **증가**.

⇒ **어떤 정보를 다음 상태로 전달할지 선택하는 방식**이 중요.

- **State space models**
- **RWKV**
- **Mamba**
- **Simplified State Space Layers for Sequence Modeling**

강조점

1. **RNN은 상태를 갱신하며 시퀀스를 처리**하고, 시간에 따라 같은 파라미터를 공유.
2. **BPTT는 시간축으로 펼친 계산 그래프의 역전파**. Truncated BPTT는 상태 전달을 유지하면서 역전파 길이를 제한.
3. 언어 모델은 **다음 문자·토큰의 분포**를 예측하고, 생성할 때는 이전 출력을 다음 입력으로 사용.

4. Vanilla RNN의 반복 곱은 **기울기 소실·폭발**을 일으킬 수 있음. 폭발은 clipping으로 제어.
5. LSTM은 **셀 상태와 게이트**로 정보 유지·기록·출력을 조절하며, 장거리 정보 전달을 돕는 경로 제공.
6. 더 나은 시퀀스 구조와 상태 전달 방식은 계속 연구되는 주제.